Tarea 2 – Torneo de Komikan Concurrente

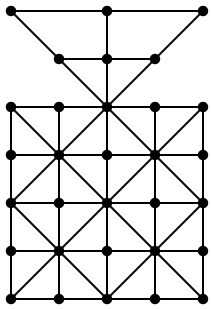
Sistemas Operativos

1. Diseño de la sincronización

Hicimos todo en un solo proceso con hilos (pthreads). Como los hilos comparten la memoria, no fue necesario usar memoria compartida de System V. Los hilos son: **main** (servidor), **N jugadores**, **un juez por mesa activa** y **13 fichas por partida** (1 puma y 12 perros).

Servidor y emparejamiento. Cada jugador que llega se anota en una cola protegida por `mcola` y hace `sem_post(&avisos)`. El servidor duerme en `sem_wait(&avisos)` y solo despierta cuando llega alguien, así que no hay espera activa. Cuando hay dos jugadores en la cola, el primero juega con el puma y el segundo con los perros. Para no pasarnos de M mesas usamos el semáforo `mesas_libres`, que parte en M: si las M mesas están ocupadas, la pareja queda bloqueada en `sem_wait` hasta que algún juez hace `sem_post`. Cada jugador espera el resultado de su partida en su propio semáforo `listo`.

Tablero. Lo guardamos en una matriz de 7×5. Las filas 2 a 6 son el alquerque de 5×5, con diagonales solo en los puntos "pares", y las filas 0 y 1 son el trono. El trono es la versión estándar del Komikan: un triángulo pegado al centro del lado superior, con una línea horizontal al medio y una vertical. Son 6 puntos más, y sus líneas están en la tabla `trono` del código. La figura de la derecha se generó desde las mismas conexiones de `conectadas()`. La revisamos con un script: todas las conexiones son simétricas y todo salto del puma (A→perro→C) va en línea recta. Si el enunciado usa otro diseño de trono, solo hay que cambiar esa tabla.



fila 0 arriba, fila 6 abajo

Tablero usado (31 puntos)

Mutex del tablero. Cada partida tiene **un mutex** que protege todo: tablero, capturas, movimientos y si terminó. Cada ficha hace esto una y otra vez:

- Toma el mutex, anota dónde está, elige un movimiento y suelta el mutex.
- Duerme entre 50 y 300 ms.
- Vuelve a tomar el mutex y llama a `mover()`, que es la parte del juez que valida el movimiento. Revisa que la ficha siga en la casilla desde donde decidió, que el destino esté vacío y que el movimiento o el salto respete las líneas del tablero. Si otra ficha ganó la casilla mientras ésta dormía, el movimiento se rechaza y queda en el log.

Como la validación y la escritura se hacen con el mutex tomado, si dos fichas quieren la misma casilla solo la primera que entra se la queda. En `mover()` también se revisa si alguien ganó. Cuando la partida termina se hace `sem_post(&p->fin)`. El juez está dormido en ese semáforo, y al despertar hace `join` de las fichas, destruye el mutex y el semáforo, avisa a los jugadores y libera la mesa.

Fallos. El mutex del tablero es *robusto* (`PTHREAD_MUTEX_ROBUST`). Para probarlo, con `prob_falla` una ficha se muere de golpe (`pthread_exit`) con el mutex tomado y sin avisar a nadie. El siguiente hilo que pide el mutex recibe `EOWNERDEAD`, lo repara con `pthread_mutex_consistent`, mira en `p->dueno` qué ficha murió y declara ganador por abandono al otro jugador. Si no usáramos esto, todas las fichas quedarían bloqueadas para siempre.

Ctrl+C. El handler de `SIGINT` solo pone `parar = 1` y hace `sem_post` en `avisos` y `mesas_libres`, que se puede hacer dentro de un handler. El servidor despierta y cancela las partidas activas. Desde ese momento, todo jugador que llega igual se anota en la cola, pero el servidor lo rechaza y le avisa, así que ninguna llegada queda sin procesar. Los jugadores que todavía no llegaban no están durmiendo: esperan en el semáforo `cierre` con `sem_timedwait`, y el handler les hace `sem_post`, así que llegan altiro, son rechazados y el cierre no tiene que esperarlos. Cuando ya no quedan jugadores por emparejar, el servidor sigue durmiendo en `avisos` hasta que terminen todas las partidas. Cada juez hace `sem_post` al salir, así que un Ctrl+C en ese momento también cancela las partidas al instante. Al final se hace `join` de todos los

hilos, se destruyen los semáforos y los mutex, se hace `free` de todo y se cierra el log. Además se revisa que funcionen los `malloc`, los `pthread_create` y los `sem_init`. Si `sem_wait` se corta por la señal (`EINTR`) se reintenta, y cualquier otro error termina el programa.

2. Riesgos de concurrencia

Deadlock: con un mutex por casilla, el salto del puma necesitaría tres casillas y dos fichas podrían quedar esperándose entre ellas. Por eso usamos un solo mutex por tablero. Cuando se toman dos mutex a la vez siempre es en el mismo orden: `mmesas` → partida → `mlog`. Ninguna ficha duerme con el mutex tomado. El caso de un hilo que muere con el mutex tomado lo resuelve el mutex robusto.

Livelock: las fichas podrían moverse para siempre sin que nadie gane, o pelear siempre por la misma casilla. Lo segundo es muy poco probable porque los retardos son aleatorios. Para lo primero hay un límite de movimientos (`max_movimientos`): al llegar al límite la partida es empate y la mesa se libera.

Starvation: cada ficha suelta el mutex altiro y luego duerme, así que las demás siempre tienen su oportunidad. Las parejas salen de la cola en orden de llegada y solo el servidor espera mesas, así que ninguna pareja se queda esperando para siempre.

3. Evaluación

Elegimos lo más simple que funcionara bien. Un mutex por tablero es fácil de programar y de explicar, y no hay que preocuparse del orden de candados entre casillas. Lo malo es que dentro de una mesa se mueve una sola ficha a la vez, pero la sección crítica es muy corta (validar, mover y escribir en el log) comparada con los 50–300 ms que duerme cada ficha, así que casi nunca hay que esperar el mutex y la latencia de un movimiento es casi solo el retardo aleatorio. Las mesas no comparten el mutex, así que sí juegan en paralelo de verdad. Todas las esperas del servidor, de los jueces y de los jugadores son con semáforos, por lo que no se gasta CPU esperando.

Limitaciones.

- **Abandono:** solo se detecta si la ficha muere con el mutex tomado, que es justo lo que simula `prob_falla`. Si un hilo terminara de golpe sin tener el mutex, nadie se enteraría. Para cubrir eso haría falta algo como un "latido" que el juez revise, y no lo implementamos.
- **Errores graves:** al ser hilos de un mismo proceso, un segmentation fault en una ficha bota todo el programa. Para aguantarlo habría que usar procesos con `fork`.
- **Semilla:** fija la llegada de los jugadores y las decisiones de cada ficha (cada hilo tiene su propia semilla para `rand_r`), pero no da reproducibilidad total. El orden en que las fichas consiguen el mutex lo decide el scheduler del sistema operativo, así que dos corridas con la misma semilla pueden terminar distinto.